Code Explanation

Data Processing Test Suite Explanation

This test suite is designed to validate the functionality of data processing functions, specifically `clean_data` and `process_order_data`, using PySpark. The tests cover data cleaning and processing for customer and order data.

Overview of the Code
The provided code is a test suite written in Python using the unittest framework. It tests two main functions: `clean_data` and `process_order_data`. The `clean_data` function is expected to remove null values and duplicates from the customer data, while the `process_order_data` function is expected to add a `TotalAmount` column to the order data by calculating the product of `PricePerUnit` and `Qty`.

Step 1: Setting Up the Test Environment
This step involves creating a Spark session and sample data for testing. The sample data includes customer and order information with predefined schemas.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestDataProcessing")
        .master("local[*]")
        .getOrCreate()
```

Step 2: Creating Sample Customer Data
In this step, a schema for customer data is defined, and sample customer data is created. The data includes null values and duplicates to test the `clean_data` function.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

cls.customer_data = [
    ("C001", "John Doe", "john@example.com", "North"),
    ("C002", "Jane Smith", "jane@example.com", "South"),
    ("C003", "Bob Johnson", "bob@example.com", "East"),
    ("C004", "Alice Brown", "alice@example.com", "West"),
    ("C005", None, "mike@example.com", "North"),  # Null value
    ("C001", "John Doe", "john@example.com", "North")  # Duplicate
]

cls.customer_df = cls.spark.createDataFrame(cls.customer_data, custo
mer_schema)
```

Step 3: Creating Sample Order Data
This step involves defining a schema for order data and creating sample order data. The data includes null values and duplicates to test the `process_order_data` function.

```
order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])

cls.order_data = [
    ("O001", "Laptop", 1200.0, 1, datetime.date(2023, 1, 15), "C001"
),
    ("O002", "Phone", 800.0, 2, datetime.date(2023, 1, 20), "C002"),
    ("O003", "Headphones", 100.0, 3, datetime.date(2023, 2, 5), "C00
3"),
    ("O004", "Monitor", 300.0, 1, datetime.date(2023, 2, 10), "C004"
),
    ("O005", "Keyboard", 50.0, None, datetime.date(2023, 3, 1), "C00
1"),  # Null value
    ("O001", "Laptop", 1200.0, 1, datetime.date(2023, 1, 15), "C001"
)  # Duplicate
]

cls.order_df = cls.spark.createDataFrame(cls.order_data, order_schem
a)
```

Step 4: Testing the `clean_data` Function
This step tests the `clean_data` function by verifying that it removes null values and duplicates from the customer data.

```
def test_clean_data(self):
    clean_customer_df = clean_data(self.customer_df)
    self.assertEqual(clean_customer_df.count(), 4)
    self.assertEqual(clean_customer_df.filter(clean_customer_df.Cust
Id == "C001").count(), 1)
```

Step 5: Testing the `process_order_data` Function
This step tests the `process_order_data` function by verifying that it adds a `TotalAmount` column to the order data and calculates the total amount correctly.

```
def test_process_order_data(self):
    processed_order_df = process_order_data(self.order_df)
    self.assertTrue("TotalAmount" in processed_order_df.columns)
    row = processed_order_df.filter(processed_order_df.OrderId == "O
002").collect()[0]
    self.assertEqual(row.TotalAmount, 1600.0)  # 800.0 * 2 = 1600.0
```

Step 6: Tearing Down the Test Environment
This final step involves stopping the Spark session after the tests are completed.

```
@classmethod
def tearDownClass(cls):
    cls.spark.stop()
```